# faultbox: Cost and Coverage of In-Process Fault Isolation with Memory Protection Keys

Meghashyam Sai Dontha, Blekinge Institute of Technology (BTH)

November 2026

**Code and data:** https://github.com/saishyamdontha/faultbox

## Abstract

Systems that run untrusted code inside a host process, such as plugin hosts and language runtimes, depend on components outside any sandbox, and a fault in one of them can crash or corrupt the whole process. This report compares six ways for a C host to call untrusted plugins: no isolation, two forms of process isolation (a new `seccomp`-sandboxed process per call, and a persistent sandboxed worker), and three variants of in-process isolation using x86 memory protection keys (MPK/PKU) with signal-based recovery. Against nine faulty or malicious plugins, process isolation keeps the host intact in all nine cases and MPK isolation in six. MPK fails against writes to unprotected memory, `exit()`, and a plugin that executes `WRPKRU`. In exchange, the MPK domain switch adds about 21 ns per call, roughly 225x less than a persistent sandboxed worker (5.5 µs). A naive MPK implementation costs 314 ns per call, about 92% of which is one signal-mask syscall per call rather than the key switch. A load-time scanner blocks the direct `WRPKRU` bypass but not indirect ones.

## 1. Motivation

Process isolation is the standard way to contain untrusted code, but every call then crosses a process boundary. In-process isolation keeps calls cheap by dividing one address space into memory domains. MPK supports this in hardware: each page carries a 4-bit key, and a per-thread register (PKRU) controls access to each key, changed with the unprivileged `WRPKRU` instruction.

This project asks three questions:

1. Which faults does each strategy actually contain?
2. What does each strategy cost per call and per recovery?
3. Where does MPK-based isolation stop working as a boundary?

## 2. Threat and fault model

The host owns two pieces of memory: `host_state`, one page tagged with a protection key in MPK modes, and `host_config`, an ordinary global that is never protected. Plugins are shared objects loaded with `dlopen` that export `plugin_run(int)`.

| Plugin | Behaviour | Class |
|---|---|---|
| `good` | Returns `2 * input` | control |
| `nullderef` | Null pointer dereference | crash (SIGSEGV) |
| `oob_write` | Writes into `host_state` | corruption (protected) |
| `corrupt_unprotected` | Writes into `host_config` | corruption (unprotected) |
| `divzero` | Integer division by zero | crash (SIGFPE) |
| `illegal` | Executes `ud2` | crash (SIGILL) |
| `abort_call` | Calls `abort()` | crash (SIGABRT) |
| `stack_overflow` | Unbounded recursion | crash (guard page) |

| Plugin | Behaviour | Class |
|---|---|---|
| `exit_call` | Calls `exit(42)` | process termination |
| `wrpkru_attack` | Sets PKRU = 0 with `WRPKRU`, writes `host_state` | malicious bypass |

The primary model is a **buggy** plugin. The malicious plugin tests where the boundary fails; the project does not claim protection against hostile code.

## 3. Design

| Mode | Isolation | Recovery |
|---|---|---|
| `none` | None | None |
| `fork` | New process per call, `seccomp` strict, result via pipe | Child dies; host continues |
| `ipc` | Persistent worker under `seccomp` strict; pipe round-trip per call | Worker reaped and respawned |
| `mpk` | `host_state` key disabled while the plugin runs | Handler `siglongjmp`s back, restores PKRU, reloads the plugin |
| `mpk_fast` | As `mpk`, without saving the signal mask | As `mpk` |
| `mpk_scan` | As `mpk_fast`, plus a load-time scan for `WRPKRU` (`0F 01 EF`) | As `mpk`; flagged plugins never load |

Four implementation details turned out to matter:

- **PKRU after recovery.** The kernel resets PKRU on signal delivery, and `siglongjmp` skips `sigreturn`, so the host restores access to its own key explicitly.
- **Signal mask.** `sigsetjmp(buf, 1)` saves the signal mask with a syscall on every call. Skipping the save is safe only with an `SA_NODEFER` handler; otherwise the signal stays blocked after the jump, and the next fault kills the process.
- **Alternate signal stack.** A stack-overflowing plugin leaves no stack for the handler. Installing the handler with `SA_ONSTACK` on a `sigaltstack` makes this fault recoverable.
- **Strict seccomp exit.** glibc's `_exit()` uses `exit_group`, which strict mode forbids, so sandboxed children exit with `syscall(SYS_exit)`. The host ignores SIGPIPE so that writing to a dead worker cannot kill it.

## 4. Evaluation setup

- **Hardware/OS:** AMD Ryzen 9 8940HX (PKU supported), Ubuntu 24.04, gcc. Host built with `-O2`, plugins with `-O0` so the fault code is kept as written.
- **Containment:** each (mode, plugin) pair runs in a fresh host process; the outcome is classified from the exit status and checks of both host memory regions.
- **Per-call cost:** calls to `good` after 1,000 warm-up calls, timed with `CLOCK_MONOTONIC`: 1,000,000 calls for `none` and MPK modes, 100,000 for `ipc`, 10,000 for `fork`.
- **Fault + recovery:** mean over 1,000 calls to `nullderef`, including plugin reload (MPK) or worker respawn (`ipc`).
- **Repetitions:** 5 runs per mode, pinned to one core with `taskset`; means reported.

## 5. Results

### 5.1 Containment

| Plugin | none | fork | ipc | mpk, mpk_fast | mpk_scan |
|---|---|---|---|---|---|
| nullderef | crashed | contained | contained | contained | contained |
| oob_write | corrupted | ok* | ok* | contained | contained |
| corrupt_unprotected | corrupted | ok* | ok* | **corrupted** | **corrupted** |
| divzero | crashed | contained | contained | contained | contained |
| illegal | crashed | contained | contained | contained | contained |
| abort_call | crashed | contained | contained | contained | contained |
| stack_overflow | crashed | contained | contained | contained | contained |
| exit_call | host exited | contained | contained | **host exited** | **host exited** |
| wrpkru_attack | corrupted | ok* | ok* | **corrupted** | rejected |
| **Host intact (of 9)** | **0** | **9** | **9** | **6** | **7** |

* The write landed in the worker's copy-on-write memory. The control plugin `good` succeeded in every mode.

In the MPK modes, only `oob_write` is stopped by the protection key itself (`SEGV_PKUERR`). The other five crash faults are survived because of the signal handler and the alternate stack, not because of MPK.

### 5.2 Performance

| Mode | Per call | Overhead vs `none` | Fault + recovery |
|---|---|---|---|
| none | 3.3 ns | – | – |
| mpk_fast | 24.4 ns | +21 ns | 30.1 µs |
| mpk | 314 ns | +311 ns | 31.1 µs |
| ipc | 5,508 ns | +5.5 µs | 125.1 µs |
| fork | 113,838 ns | +114 µs | 111.7 µs |

## 6. Discussion

**Cost versus coverage.** Against the realistic process-isolation baseline, a persistent sandboxed worker, MPK isolation is about 225x cheaper per call, but it contains 6 rather than 9 of the tested fault classes. Its failures are structural: it protects only memory explicitly tagged with a key, and it does not restrict what the plugin may ask the kernel to do.

**The domain switch is cheap; bookkeeping is not.** The naive MPK mode spends about 92% of its overhead on one `rt_sigprocmask` syscall per call (confirmed with `strace`) caused by saving the signal mask. For in-process isolation, the machinery around each call can cost an order of magnitude more than the isolation itself, so it must be designed and measured as carefully as the boundary.

**Recovery.** MPK recovers in about 30 µs versus about 125 µs for respawning a worker. The MPK figure is probably dominated by `dlclose`/`dlopen`; recovering by restoring a saved domain snapshot instead of reloading code could reduce it further. These breakdowns are inferences and have not been measured separately.

**MPK as a security boundary.** Because `WRPKRU` is unprivileged, code in the plugin domain can unlock every key. The scanner blocks the direct case, but a plugin can still call glibc's `pkey_set()`.

**Implication for runtimes that host untrusted code.** Language runtimes such as WebAssembly engines already sandbox guest code, but host functions, system-interface implementations and third-party libraries run outside that sandbox. These results suggest in-process domains are cheap enough to place around such components on every call. They also suggest the hard parts are coverage (protecting all relevant memory and restricting syscalls), recovery design and resistance to `WRPKRU` misuse, not the switch cost.

## 7. Limitations and future work

- MPK modes protect one page and filter no syscalls.

- The scanner misses library calls (`pkey_set`), `XRSTOR` and code generated at runtime; it can flag chance byte matches, and it scans by path before loading (a time-of-check/time-of-use window).
- Single-threaded; no hang detection; reload does not roll back external side effects; repeated `abort()` recovery within one process is untested.
- One machine, means only, CPU frequency not fixed.

**Future work:** measure the recovery breakdown; combine MPK with a syscall filter; give each plugin its own key so co-located plugins are isolated from each other; scan through the opened file descriptor to close the TOCTOU window; reject plugins importing `pkey_*` symbols; evaluate on Intel CPUs.

## 8. Conclusion

On this machine, MPK-based in-process isolation contained every crash-type fault tested at about 21 ns per call, roughly 225x cheaper than a persistent sandboxed worker. It did not contain writes to unprotected memory, `exit()`, or a plugin executing `WRPKRU`, all of which process isolation handled. Most of the cost of a naive MPK implementation came from signal-mask handling rather than the key switch.

## References


1. A. Vahldiek-Oberwagner et al., "ERIM: Secure, Efficient In-process Isolation with Protection Keys (MPK)," USENIX Security Symposium, 2019.
2. M. Hedayati et al., "Hodor: Intra-Process Isolation for High-Throughput Data Plane Libraries," USENIX Annual Technical Conference, 2019.
3. M. Gülmez et al., "Rewind & Discard: Improving Software Resilience using Isolated Domains," IEEE/IFIP DSN, 2023.
4. Linux man-pages: `pkeys(7)`, `seccomp(2)`, `sigaction(2)`, `sigaltstack(2)`, `sigsetjmp(3)`.